# SIMATS ENGINEERING

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES**

**CHENNAI-602105**

| | |
|---|---|
| NAME | AKSHAYA R S |
| COURSE CODE | CSA0209 |
| REG NO | 1924260209 |
| GITHUB LINK | https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING1924260209 |

## COURSE OUTCOME 2

# LOGIC ANALYSIS AND DESIGN CHALLENGES

## Q1 – Precedence, Associativity and the Discount Bug (6 Marks)

### 1. Aim

The aim is to understand how operator precedence and associativity affect C expressions, identify errors in a pharmacy billing calculation, understand integer division and undefined behaviour, and rewrite the calculation in a clear and correct form.

### 2. Problem Analysis

The pharmacy must calculate gross amount (price × quantity), apply a 15% senior-citizen discount, add 5% GST, and then subtract Rs 50 if the resulting amount exceeds Rs 1,000. The developer wrote:

```
total = price * qty - price * qty * 15 / 100 * senior
+ price * qty * 5 / 100; net = total > 1000 ? total -
50 : total; count = count++ + 1;
```

The first expression is grouped by C precedence rules. If the variables are integers, integer division discards fractional parts. The last statement has undefined behaviour because it modifies count more than once without the required sequencing.

### 3. Explanation

### 3.1 Operator precedence and associativity

| Operators | Precedence (relative) | Associativity |
|---|---|---|
| () grouping | Highest | Determined by grouping |
| * / % | High | Left to right |
| + - | Below multiplicative | Left to right |
| < <= > >= | Below additive | Left to right |
| == != | Below relational | Left to right |
| && | Below equality | Left to right |
| \|\| | Below && | Left to right |
| ?: | Low | Right to left |
| = | Very low | Right to left |

Precedence determines how operators are grouped. Associativity determines grouping when operators have equal precedence. It does not mean every operand is necessarily evaluated from left to right.

**Example:** `10 + 5 * 2` is grouped as `10 + (5 * 2)`, giving 20. `100 / 10 * 2` is grouped as `(100 / 10) * 2`, giving 20.

## 3.2 Grouping of the original billing expression

```
total = (price * qty)       - (((price * qty) *
15 / 100) * senior)
      + ((price * qty) * 5 / 100);
```

The arithmetic grouping is: calculate gross; multiply gross by 15 and divide by 100; multiply that discount by senior; calculate GST by multiplying gross by 5 and dividing by 100; subtract discount and add GST. Here senior is assumed to be 1 for a senior citizen and 0 otherwise.

## 3.3 Manual predictions before execution

| Example | Price | Qty | Senior | Gross | Discount | GST | Total | Net |
|---|---|---|---|---|---|---|---|---|
| 1 | 120 | 10 | 1 | 1200 | 180 | 60 | 1080 | 1030 |
| 2 | 80 | 10 | 1 | 800 | 120 | 40 | 720 | 720 |
| 3 | 100 | 10 | 0 | 1000 | 0 | 50 | 1050 | 1000 |

**Example 1:** Gross = 120 × 10 = Rs 1,200. Discount = 1,200 × 15 / 100 = Rs 180. GST = 1,200 × 5 / 100 = Rs 60. Total = 1,200 − 180 + 60 = Rs 1,080. Since this exceeds Rs 1,000, net = 1,080 − 50 = Rs 1,030.

**Example 2:** Gross = Rs 800; discount = Rs 120; GST = Rs 40; total = Rs 720. The extra Rs 50 discount does not apply.

**Example 3:** Gross = Rs 1,000; senior = 0, so discount = 0; GST = Rs 50; total = Rs 1,050; net = Rs 1,000.

## 3.4 Integer division

When both operands of / are integers, C performs integer division and discards the fractional part. For example, `15 / 100` produces 0, not 0.15. In the original expression, multiplication happens first, so 1,200 × 15 / 100 gives 180. But for gross = 1,001, 1,001 × 15 / 100 gives 150, losing Rs 0.15. Similarly, 1,001 × 5 / 100 gives 50, losing Rs 0.05. The integer expression therefore gives 901, while the exact arithmetic result before currency rounding is 900.90.

The original expression's main billing issue is not that precedence is ambiguous; it is that integer arithmetic truncates fractional currency amounts and the long expression is difficult to audit.

## 3.5 Conditional operator

The conditional operator has the form `condition ? expression1 : expression2`. In `net = total > 1000 ? total - 50 : total;`, the amount is reduced by Rs 50 only when total is greater than Rs 1,000. It is equivalent to:

```
if (total > 1000)
net = total - 50; else
net = total;
```

## 3.6 Undefined behaviour in count++

The statement `count = count++ + 1;` modifies count through the post-increment and also through assignment, without a sequencing rule that makes the modifications well-defined. The C standard therefore gives no guaranteed result. Do not claim it always produces a particular number.

Use `count = count + 1;` or simply `count++;`. GCC's `-Wall` and `-Wextra` enable many warnings and may diagnose this suspicious expression. Exact warning text depends on compiler version. A warning does not fix the code.

## 4. Prediction Before Execution

Expected integer results for the three selected examples are: (1) total 1080, net 1030; (2) total 720, net 720; (3) total 1050, net 1000. The corrected floating-point program below retains cents and displays two decimal places.

## 5. C Program / Implementation

### 5.1 Original integer-expression demonstration

```
#include <stdio.h>

int main(void) {
    int price, qty, senior;
    int total, net;

    price = 120;
    qty = 10;
    senior = 1;
    total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    net = total > 1000 ? total - 50 : total;
    printf("Example 1: total = %d, net = %d\n", total, net);
    price = 80;
    qty = 10;
    senior = 1;
    total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    net = total > 1000 ? total - 50 : total;
    printf("Example 2: total = %d, net = %d\n", total, net);

    price = 100;
    qty = 10;
    senior = 0;
    total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    net = total > 1000 ? total - 50 : total;
    printf("Example 3: total = %d, net = %d\n", total, net);
    return 0;
}
```

### 5.2 Corrected and readable billing program

```
#include <stdio.h>
int main(void) {
    double price, gross, discount, tax, total, net;
    int qty, senior;

    printf("Enter price per item: ");
    if (scanf("%lf", &price) != 1) return 1;

    printf("Enter quantity: ");
    if (scanf("%d", &qty) != 1) return 1;
    printf("Is the customer a senior citizen? (1-Yes, 0-No): ");
    if (scanf("%d", &senior) != 1) return 1;

    if (price < 0 || qty < 0 || (senior != 0 && senior != 1)) {
        printf("Invalid input\n");
        return 1;
    }

    gross = price * qty;
    discount = senior ? (gross * 15.0 / 100.0) : 0.0;
    tax = gross * 5.0 / 100.0;
    total = (gross - discount) + tax;
    net = (total > 1000.0) ? (total - 50.0) : total;

    printf("\nGross amount = Rs %.2f\n", gross);
    printf("Senior discount = Rs %.2f\n", discount);
    printf("GST = Rs %.2f\n", tax);
    printf("Total = Rs %.2f\n", total);
    printf("Net amount = Rs %.2f\n", net);

    return 0;
}
```

The corrected program uses intermediate variables for gross, discount, tax, total and net. It checks input conversion and validates the quantity, price and senior flag. It uses double for this introductory example; a production billing system should preferably store money as integer paise and define a rounding rule.

## 6. Sample Input and Expected Output

**Input:**

```
120
10
1
```

**Expected output:**

```
Gross amount = Rs 1200.00
Senior discount = Rs 180.00
GST = Rs 60.00
Total = Rs 1080.00
Net amount = Rs 1030.00
```

Second example input: price 100, quantity 10, senior 0. Expected gross = Rs 1,000.00, discount = Rs 0.00, GST = Rs 50.00, total = Rs 1,050.00 and net = Rs 1,000.00.

## 7. Output Explanation

For the first input, gross = 120 × 10 = 1,200. The discount is 180 and GST is 60. The amount after discount and GST is 1,080. Because 1,080 is greater than 1,000, the additional Rs 50 is deducted. The net amount is Rs 1,030.00.

## 8. Alternative Concept and Complexity Analysis

Using separate statements gives gross = price × quantity; discount = senior ? gross × 15 / 100 : 0; tax = gross × 5 / 100; total = gross  discount + tax; and net = total > 1000 ? total  50 : total.

| Factor | One long expression | Intermediate variables |
|---|---|---|
| Readability | Less clear | Clearer |
| Debugging | Harder to isolate errors | Each amount can be checked |
| Temporary storage | Fewer named values | A few additional variables |
| Time complexity | O(1) | O(1) |
| Space complexity | O(1) | O(1) |

Both approaches perform a fixed number of arithmetic operations for one bill, so both have O(1) time complexity. The additional variables are fixed in number, so auxiliary space remains O(1). Intermediate variables improve correctness review and readability. The recommended design uses both parentheses and intermediate variables.

## 9. Evidence / Result

Include: (1) three manual predictions; (2) operator grouping explanation; (3) original program and GCC execution screenshot; (4) corrected source and output screenshot; (5) integer-division example; (6) explanation of undefined behaviour; and (7) GCC compilation using `-Wall -Wextra`. `gcc -std=c11 -Wall -Wextra -o q1 q1.c ./q1`

# Q2 – When Does Short-Circuit Evaluation Matter? (4 Marks)

## 1. Aim

The aim is to understand short-circuit evaluation in C and determine how condition order changes calls to a slow verification function. It also compares logical and bitwise operators and considers the effect of audit-log side effects.

## 2. Problem Analysis

A concession is granted when the passenger is both a senior citizen and travelling in an eligible class, OR when the passenger has a valid disability card:

```
(senior && eligible) || verify_card(disability)
```

The function verify_card() represents a slow server verification. Its call count can be reduced when the first part of the OR expression is already true.

## 3. Explanation

### 3.1 Short-circuit evaluation of &&

For `A && B`, if A is false, C skips B because the final result must be false. Example: `if`

```
(a != 0 && b / a > 2) {
printf("Valid"); }
```

If a is zero, the first condition is false and the division expression is not evaluated. This prevents division by zero in this example.

### 3.2 Short-circuit evaluation of ||

For `A || B`, if A is true, C skips B because the final result must be true. Example:

```
if (already_eligible || verify_card(disability)) {       printf("Concession
granted"); }
```

If already_eligible is true, verify_card() is not called.

### 3.3 Difference between & and &&

| Logical && | Bitwise & |
|---|---|
| Used to combine logical conditions | Performs bit-by-bit AND |
| Short-circuit evaluation | Both operands are evaluated |
| Result is 0 or 1 | Result is a bitwise integer value |

The same distinction applies to logical OR (||) and bitwise OR (|). Bitwise operators should not be substituted for logical operators when short-circuit behaviour is required.

### 3.4 Designed experiment: 1,000 passengers

Assumptions for a reproducible hypothetical experiment (not measured railway statistics): 150 passengers are both senior and in an eligible class; 50 are senior but not eligible; 800 are non-senior. Of the 1,000 passengers, 80 disability-card holders are placed outside the group of 150 senior-and-eligible passengers. Thus the categories do not overlap.

| Passenger category | Count |
|---|---|
| Senior and eligible | 150 |
| Senior but not eligible | 50 |
| Non-senior | 800 |
| Total | 1,000 |

| Passenger category | Count |
|---|---|
| Valid disability cards (within non-qualifying group) | 80 |

**Order 1 – fast condition first:** `(senior && eligible) || verify_card(disability)`. For the 150 passengers whose first condition is true, the server function is skipped. It is called for the remaining 850. Expected calls = 1,000  150 = 850.

**Order 2 – slow condition first:** `verify_card(disability) || (senior && eligible)`. The verification function is evaluated first for every passenger. Expected calls = 1,000.

The first order saves 150 calls, which is 150 / 1,000 × 100 = 15% of the calls. Both orders grant concessions to 150 + 80 = 230 passengers.

### 3.5 Cheapest / most-likely condition first

For AND (&&), put a cheap or likely-false condition first because false makes the whole expression false and skips later checks. For OR (||), put a cheap or likely-true condition first because true makes the whole expression true and skips later checks. The order matters less when both conditions are cheap, have no side effects and have negligible cost. Conditions must still represent the same logical rule.

## 4. Prediction Before Execution

| Condition order | Expected slow calls |
|---|---|
| Fast senior/eligible condition first | 850 |
| Slow verification first | 1,000 |
| Difference | 150 fewer calls |

These counts are derived from the assumed data before running the program. They are predictions, not measured server timings.

## 5. C Program / Implementation

```
#include <stdio.h> #define

PASSENGERS 1000 int

verification_calls = 0;

int verify_card(int disability) {
verification_calls++;     return disability;  // Simulated
verification result }

int main(void) {     int
senior[PASSENGERS] = {0};     int eligible[PASSENGERS]
= {0};     int disability[PASSENGERS] = {0};     int
concession_count = 0;
```

```
    // 150 senior passengers with eligible tickets     for
(int i = 0; i < 150; i++) {         senior[i] =
1;         eligible[i] = 1;     }

    // 50 senior passengers with ineligible tickets     for
(int i = 150; i < 200; i++) {         senior[i]
= 1;         eligible[i] = 0;     }

    // 80 disability-card holders, outside first group
for (int i = 200; i < 280; i++) {         disability[i] =
1;
    }

    // Experiment 1: fast condition first
verification_calls = 0;     concession_count
= 0;

    for (int i = 0; i < PASSENGERS; i++) {
if ((senior[i] && eligible[i]) ||
verify_card(disability[i])) {
concession_count++;         }     }

    printf("Fast condition first:\n");
printf("Verification calls = %d\n", verification_calls);
printf("Concessions granted = %d\n\n", concession_count);

    // Experiment 2: slow condition first
verification_calls = 0;     concession_count
= 0;

    for (int i = 0; i < PASSENGERS; i++) {
if (verify_card(disability[i]) ||             (senior[i]
&& eligible[i])) {             concession_count++;
}     }

    printf("Slow condition first:\n");
printf("Verification calls = %d\n", verification_calls);
printf("Concessions granted = %d\n", concession_count);
return 0;
}
```

The function is a local simulation, not a real server request. It increments a counter each time it is called. The two experiments use the same passenger arrays so that only the condition order changes.

## 6. Sample Input and Expected Output

No keyboard input is required; the program builds the assumed 1,000-passenger data internally.

```
Fast condition first:
Verification calls = 850 Concessions
granted = 230

Slow condition first:
Verification calls = 1000
Concessions granted = 230
```

## 7. Output Explanation

In the fast-first expression, the first condition is true for 150 passengers. For these passengers, the right operand of || is skipped. The verification function is called for the other 850. In the slow-first expression, verify_card() is the first operand, so it is called 1,000 times. Both approaches grant 230 concessions because the 80 disability-card holders are outside the first qualifying group.

## 7.1 Changed condition: verification records an audit log

If verify_card() also records every passenger in an audit log, skipping the function for 150 passengers would leave missing audit records. In that case, call it separately for every passenger:

```
int card_valid = verify_card(disability[i]);
if ((senior[i] && eligible[i]) || card_valid) {
printf("Concession granted\n"); }
```

This makes exactly one verification call per passenger (1,000 calls) and preserves the audit requirement. Correctness and mandatory logging take priority over reducing calls.

## 8. Alternative Concept and Complexity Analysis

### 8.1 Nested if statements

```
if (senior && eligible) {
printf("Concession granted"); } else {     if
(verify_card(disability)) {
printf("Concession granted");     }
}
```

This preserves short-circuit behaviour: verification is called only when the senior-and-eligible condition is false. Under the stated data, there are 850 calls.

### 8.2 Non-short-circuit operators

```
if ((senior & eligible) | verify_card(disability)) {
printf("Concession granted"); }
```

Both operands are evaluated, so verify_card() is called for all 1,000 passengers. For 0/1 condition values the truth result may match the logical expression, but the evaluation behaviour is different.

| Approach | Calls for 1,000 passengers |
|---|---|
| Logical operators, fast condition first | 850 |
| Logical operators, slow condition first | 1,000 |
| Nested if (fast condition first) | 850 |
| Bitwise & and \| | 1,000 |
| Separate mandatory audit verification | 1,000 |

### 8.3 Time and space complexity

The outer loop processes n passengers, so the loop contributes O(n) work. If the slow function costs V time per call and is called C times, a useful cost model is O(n + C·V). If verification is treated as constant-time, overall time is O(n). With network calls, actual elapsed time also depends on network latency.

The illustrative program stores three arrays of 1,000 integers, so its sample storage is proportional to the passenger count: O(n). A streaming implementation that processes one passenger at a time needs only a fixed number of variables and has O(1) auxiliary space.

### 8.4 Advantages and disadvantages

**Short-circuit logical operators:** reduce unnecessary function calls and can avoid unsafe operations, but may skip side effects. **Nested if:** makes the evaluation sequence explicit and can preserve short-circuit behaviour, but may be longer. **Bitwise operators:** are useful for bit manipulation, but do not short-circuit and can cause unnecessary calls when used in conditions.

### 8.5 Final design decision

Use logical operators with the cheap condition first when verification has no mandatory side effect. If every passenger must be recorded, call the verification function separately before testing the concession rule. This design considers both performance and audit correctness.

## 9. Evidence / Result

Include: (1) the assumed distribution of 1,000 passengers; (2) explanation of skipped operands; (3) source code; (4) actual execution screenshot showing 850 and 1,000 calls; (5) comparison of && with & and || with |; (6) nested-if comparison; (7) audit-log redesign; and (8) time and space analysis. `gcc -std=c11 -Wall -Wextra -o q2 q2.c ./q2`

# Q3 – Smart Home Device Register Using Bitwise Operators (6 Marks)

## 1. Concept and objective

A smart home controller uses an 8-bit register to store the ON/OFF status of eight devices. Each bit represents one device: 1 means ON and 0 means OFF. Bitwise operators allow the program to change or inspect one device without affecting the other bits.

## 2. Bit positions

| Bit position | Device |
|---|---|
| 0 | PUMP |
| 1 | AC |
| 2 | LIGHT |
| 3 | FAN |
| 4 | TV |
| 5 | HEATER |
| 6 | WASHING MACHINE |
| 7 | OTHER DEVICE |

The device names are illustrative. If the assignment specifies a different mapping, use its exact bit positions.

## 3. Completed C program

```
#include <stdio.h> unsigned char

setBit(unsigned char reg, int dev) {

return reg | (1U << dev); } unsigned char

clearBit(unsigned char reg, int dev) {

return reg & ~(1U << dev); } unsigned char

toggleBit(unsigned char reg, int

dev) {     return reg ^ (1U << dev); }

int testBit(unsigned char reg, int
dev) {     return (reg & (1U << dev))

!= 0; } void printBinary(unsigned

char reg)

{
    for (int i = 7; i >= 0; i--)
printf("%d", (reg >> i) & 1U);

printf("\\n");

}

int main() {     unsigned

char reg = 0;

    reg = setBit(reg, 0);
reg = setBit(reg, 2);

printBinary(reg);     reg =
```

```
clearBit(reg, 0);     reg =
setBit(reg, 1);
printBinary(reg);
    if (testBit(reg, 1))
printf("AC is ON, ");
else         printf("AC is OFF,
");
    if (testBit(reg, 0))
printf("PUMP is ON\\n");
else         printf("PUMP is
OFF\\n");     return 0;
}
```

## 4. Explanation of bitwise operations

| Operation | Expression | Purpose |
|---|---|---|
| Set bit | reg \| (1U << dev) | Turns selected bit ON. |
| Clear bit | reg & ~(1U << dev) | Turns selected bit OFF. |
| Toggle bit | reg ^ (1U << dev) | Changes 0 to 1 or 1 to 0. |
| Test bit | reg & (1U << dev) | Checks whether selected bit is ON. |

**Set:** OR makes the selected bit 1. **Clear:** AND with an inverted mask makes the selected bit 0. **Toggle:** XOR changes only the selected bit. **Test:** AND checks whether the selected bit is set.

## 5. Manual prediction of register states

| Step | Operation | Register |
|---|---|---|
| Initial | Register starts at 0 | 00000000 |
| 1 | Set bit 0 | 00000001 |
| 2 | Set bit 2 | 00000101 |
| 3 | Clear bit 0 | 00000100 |
| 4 | Set bit 1 | 00000110 |

The final register is 00000110. Bit 1 (AC) is ON and bit 0 (PUMP) is OFF.

## 6. Expected output

```
00000101
00000110
AC is ON, PUMP is OFF
```

## 7. Important analysis questions

**Why is reg & (1U << dev) correct?** The expression first shifts 1 to the required bit position, then ANDs it with the register. This checks the selected bit.

**Why is (reg & 1) << dev incorrect?** reg & 1 checks only bit 0. Shifting that result afterwards cannot inspect another bit in the original register.

**Why use unsigned char?** An unsigned 8-bit value represents 0 to 255. A signed char may not represent all possible register patterns as positive values.

**How can the program count ON devices?**

```
int count = 0; for (int i =
0; i < 8; i++) {     if
(testBit(reg, i))
count++;
```

```
}
printf("ON devices = %d\\n", count);
```

## 8. Alternative approach: character array

Instead of a bit register, the program can store one character per device:

```
char state[8] = {'0','0','0','0','0','0','0','0'};
```

| Method | Advantage | Disadvantage |
| --- | --- | --- |
| Bitwise register | Uses less memory. | Requires understanding bitwise operators. |
| Character array | Easy to understand and update. | Uses more memory. |

For one million homes, one byte per home for the register requires about 1 MB of raw storage. An eight-byte array per home requires about 8 MB, excluding additional storage overhead.

## 9. Complexity analysis

Setting, clearing, toggling or testing one bit takes O(1) time. Printing or counting all eight bits takes O(8), which is constant time. The register uses O(1) space per home.

## 10. Compilation and validation

```
gcc -std=c11 -Wall -Wextra q3.c -o q3
./q3
```

Evidence to include: a screenshot of the source code, the compilation command and the actual program output.

# Q4 – Grade Classification Using if-else and switch (6 Marks)

## 1. Concept and objective

A college grading system assigns a letter grade based on a student's marks. The program uses if-else conditions or a switch statement to classify the marks. It must also identify invalid marks.

## 2. Grade classification

| Marks | Grade |
|---|---|
| 90–100 | S |
| 80–89 | A |
| 70–79 | B |
| 60–69 | C |
| 50–59 | D |
| 0–49 | F |
| Below 0 or above 100 | Invalid (?) |

## 3. Completed C program using if-else

```
#include <stdio.h>

int main() {
int marks;     char grade;

printf("Enter marks: ");

scanf("%d", &marks);

    if (marks < 0 || marks >
100)         grade = '?';
else if (marks >= 90)
grade = 'S';     else if (marks
>= 80)         grade = 'A';
else if (marks >= 70)
grade = 'B';     else if (marks
>= 60)         grade = 'C';
else if (marks >= 50)
grade = 'D';     else
grade = 'F';     printf("Grade
= %c\n",
grade);     return 0; }
```

## 4. Completed C program using switch

A switch statement cannot directly test a range such as 90–100. Dividing marks by 10 converts the mark into a category. For example, 85 / 10 gives 8.

```
#include <stdio.h>

int main() {
int marks;     char grade;

printf("Enter marks: ");

scanf("%d", &marks);

    if (marks < 0 || marks > 100)
    {

        grade = '?';     }
else     {         switch
(marks / 10)         {
case 10:             case 9:
grade =

'S';                 break;
            case 8:
grade = 'A';                 break;
```

```
            case 7:
grade = 'B';                break;
            case 6:
grade = 'C';                break;
            case 5:
grade = 'D';                break;
            default:
grade = 'F';          }
}     printf("Grade =
%c\n",
grade);     return 0; }
```

## 5. Explanation of switch and break

The switch compares marks / 10 with its case labels. If marks are 85, the result is 8, so case 8 assigns grade A. The break statement exits the switch. Without break, execution may continue into later cases (fall-through), potentially changing the grade.

## 6. Boundary-value testing

| Input marks | Expected grade |
|---|---|
| 100 | S |
| 90 | S |
| 89 | A |
| 80 | A |
| 79 | B |
| 70 | B |
| 69 | C |
| 60 | C |
| 59 | D |
| 50 | D |
| 49 | F |
| 0 | F |
| -1 | ? |
| 101 | ? |

## 7. Sample outputs

```
Enter marks: 95 Grade
= S

Enter marks: 49 Grade
= F

Enter marks: 101 Grade
= ?
```

## 8. Scholarship classification

The grade can also be used as part of a scholarship decision. For example, students with an S grade and family income below an institution-defined limit may qualify for a full scholarship. Students with an A grade may qualify for a partial scholarship. Other students may not qualify. The institution must define the actual income limits and scholarship amounts.

## 9. Alternative approach: lookup table

A lookup table can store the grade for each mark category. Input validation must happen before using the array index.

```
char grades[] = {'F', 'F', 'F', 'F', 'F',
'D', 'C', 'B', 'A', 'S', 'S'};
if (marks >= 0 && marks <=
100)     grade = grades[marks
/ 10]; else     grade = '?';
```

## 10. Complexity analysis

| Method | Time complexity | Extra space |
|---|---|---|
| if-else | O(1) | O(1) |
| switch | O(1) | O(1) |
| Lookup table | O(1) | O(1) for this fixed table |

All three methods are efficient for a fixed set of grade categories.

## 11. Compilation and validation

```
gcc -std=c11 -Wall -Wextra q4.c -o q4
./q4
```

Evidence to include: screenshots of the source code, successful compilation, and outputs for normal, boundary and invalid marks.

# Q5 – Prime Number Testing and Loop Optimization (7 Marks)

## 1. Concept and objective

A prime number is an integer greater than 1 that has exactly two positive factors: 1 and itself. Examples include 2, 3, 5, 7 and 11. Numbers such as 4, 6, 8, 9 and 10 are not prime. The objective is to compare a basic divisorchecking algorithm with an improved square-root algorithm.

## 2. Original algorithm

The original algorithm checks every possible divisor from 2 to n - 1. If any divisor divides n exactly, the number is not prime.

```
#include <stdio.h>

int main() {     int n,
i, isPrime = 1;     int

divisions = 0;

printf("Enter a number: ");

scanf("%d", &n);

if (n <= 1)

{
        isPrime = 0;     }
else     {          for (i = 2; i
< n; i++)         {
divisions++;

            if (n % i == 0)
{                  isPrime =
0;                  break;
}
        }
}
    if (isPrime)
printf("%d is prime\n", n);     else
        printf("%d is not prime\n",

n);     printf("Division checks =

%d\n",

divisions);     return 0; }
```

## 3. Improved algorithm using square root

The improved algorithm checks divisors only up to the square root of n. If a number has a factor greater than its square root, it must have a corresponding factor smaller than its square root. Therefore, checking beyond the square root is unnecessary.

The condition i * i <= n is used below. For very large integer values, i * i can overflow; the safer equivalent for positive n is i <= n / i. `#include <stdio.h> int main()`

```
{
    int n, i, isPrime = 1;
int divisions = 0;

printf("Enter a number: ");

scanf("%d", &n);

if (n <= 1)

{
        isPrime = 0;
}     else

    {          for (i = 2; i <= n / i;
i++)         {
divisions++;
            if (n % i == 0)
{                  isPrime =
0;                  break;
}
```

```
        }
}
    if (isPrime)         printf("%d is
prime\n", n);     else         printf("%d
is not prime\n", n);     printf("Division
checks = %d\n",
divisions);     return 0; }
```

## 4. Manual prediction and comparison

For prime inputs, the original program checks nearly every integer below n. The improved program checks only as far as the square root. For composite inputs, both algorithms may stop early when they find a divisor.

| Number | Result | Original checks | Improved checks |
|---|---|---|---|
| 97 | Prime | 95 | 8 |
| 100 | Not prime | 1 | 1 |
| 7919 | Prime | 7917 | 87 |
| 1000003 | Prime | 1000001 | 999 |

For 97, square root of 97 is about 9.85. The improved program checks 2 through 9, giving 8 checks. For 100, the first divisor is 2, so both versions stop after one check.

## 5. Sample outputs

```
Enter a number: 97
97 is prime
Division checks = 8

Enter a number: 100 100
is not prime
Division checks = 1
```

## 6. Comparing for, while and do-while loops

**for loop:** Suitable when the initialization, condition and update are known. **while loop:** Checks the condition before executing and may run zero times. **do-while loop:** Executes the body at least once before checking the condition. A for loop is clear for prime testing because the divisor starts at 2 and increases to a known limit.

## 7. What happens if the condition is changed to i < n?

Changing the improved condition to i < n makes the program check nearly every possible divisor again. This removes the square-root optimization and increases the number of division checks.

## 8. Alternative approach: Sieve of Eratosthenes

If many prime numbers are required up to a fixed limit, the Sieve of Eratosthenes is more suitable. It creates a list of numbers, starts with 2, marks multiples of each prime as non-prime, and repeats for the next unmarked number. The remaining unmarked numbers are prime.

**Time complexity:** O(n log log n). **Space complexity:** O(n). It is useful for finding many primes, while the squareroot method is suitable for checking one number.

## 9. Complexity analysis

| Algorithm | Time complexity | Extra space |
|---|---|---|
| Original divisor checking | O(n) | O(1) |
| Square-root method | O(√n) | O(1) |
| Sieve of Eratosthenes | O(n log log n) | O(n) |

The square-root method significantly reduces the number of checks for large prime numbers.

## 10. Compilation and validation

```
gcc -std=c11 -Wall -Wextra q5.c -o q5
./q5
```

Test using 97, 100, 7919, 1000003, 1 and 2.

# Q6 – Loan Repayment Schedule Using a Loop (8 Marks)

## 1. Concept and objective

A loan repayment program calculates the outstanding balance every month. The borrower receives a principal amount of Rs 10,000. The monthly interest rate is 2%, and the regular monthly repayment is Rs 2,000. Interest is added to the outstanding balance before the repayment is subtracted.

## 2. Given values

| Item | Value |
|---|---|
| Principal | Rs 10,000.00 |
| Monthly interest rate | 2% = 0.02 |
| Regular monthly repayment | Rs 2,000.00 |
| Maximum permitted iterations | 360 months |

## 3. Formula used

**Monthly interest** = Opening balance × 0.02

**Amount due** = Opening balance + Monthly interest

**Closing balance** = Amount due − Payment

The final payment must not exceed the amount due. Therefore, if the amount due is less than Rs 2,000, the borrower pays only the remaining amount.

## 4. Manual calculation

| Month | Opening balance (Rs) | Interest 2% (Rs) | Payment (Rs) | Closing balance (Rs) |
|---|---|---|---|---|
| 1 | 10,000.0000 | 200.0000 | 2,000.0000 | 8,200.0000 |
| 2 | 8,200.0000 | 164.0000 | 2,000.0000 | 6,364.0000 |
| 3 | 6,364.0000 | 127.2800 | 2,000.0000 | 4,491.2800 |
| 4 | 4,491.2800 | 89.8256 | 2,000.0000 | 2,581.1056 |
| 5 | 2,581.1056 | 51.6221 | 2,000.0000 | 632.7277 |
| 6 | 632.7277 | 12.6546 | 645.3823 | 0.0000 |

The loan is cleared in 6 months. In month 6, the final payment is about Rs 645.38 rather than the regular Rs 2,000. Small differences may occur if the system rounds interest to two decimal places each month.

## 5. Complete C program

```
#include <stdio.h> int

main()

{     double balance = 10000.0;
const double rate = 0.02;     const
double repayment = 2000.0;     int
month = 0;

    if (balance <= 0 || repayment <= 0 || rate < 0)     {
printf("Invalid loan details.\\n");         return 1;
}
```

```
    if (repayment <= balance * rate)     {         printf("Repayment
must be greater than monthly interest.\\n");         return 1;

    }     printf("Month  Opening       Interest      Payment

Closing\\n");

    while (balance > 0.005 && month < 360)
{         double opening = balance;
double interest = opening * rate;
double amountDue = opening + interest;
double payment = repayment;         if
(payment > amountDue)
payment = amountDue;         balance =

amountDue - payment;         if

(balance < 0.005)

balance = 0.0;         month++;         printf("%5d  %10.2f

%10.2f  %10.2f  %10.2f\\n",               month, opening,

interest, payment, balance);     }

    if (balance <= 0.005)         printf("Loan cleared in
%d months.\\n", month);     else         printf("Loan not
cleared within 360 months.\\n");     return 0;
}
```

## 6. Explanation of the program

The balance begins at Rs 10,000. The program calculates monthly interest and adds it to the opening balance. It then subtracts the payment. The while loop repeats while a positive balance remains and the 360-month limit has not been reached.

The variable **amountDue** stores the balance after interest. If the regular payment is larger than amountDue, the program uses amountDue as the final payment. This prevents the balance from becoming negative.

## 7. Why use a while loop?

A while loop is suitable because the number of repayment months is not fixed in advance. The loop continues until the loan is cleared. A for loop could also be used with a 360-month limit, but while expresses the balance-based stopping condition clearly.

## 8. Floating-point termination and rounding

Floating-point calculations may leave a very small remainder because decimal values cannot always be represented exactly in binary. The program therefore uses a small tolerance of Rs 0.005 when checking whether the balance is cleared. A real billing or banking system should apply a defined currency-rounding policy, normally calculating and storing amounts in paise using integer arithmetic or a suitable decimal type.

## 9. Safety checks

**Payment must exceed interest:** If the repayment is equal to or less than the monthly interest, the principal does not decrease. The program checks this before entering the loop.

**Maximum 360 months:** The loop has a safety limit so it cannot run indefinitely if the input values are unsuitable.

**Final payment:** The last payment is limited to the amount due, preventing an overpayment.

## 10. What happens when repayment approaches the interest?

When the repayment is only slightly greater than the monthly interest, only a small amount reduces the principal. The loan then takes much longer to clear. If repayment is equal to or below interest, the loan will not be paid off under the same fixed conditions.

## 11. Direct mathematical formula

For a fixed payment A, principal P and monthly interest rate r, the theoretical number of payments is: n = ln(A

/ (A  rP)) / ln(1 + r)

This formula applies when A is greater than rP and the rate and payment remain constant. For this example, P = 10,000, r = 0.02 and A = 2,000. It gives approximately 5.32 payment periods, so 6 actual monthly payments are required. The last payment is smaller than the regular payment.

## 12. Complexity analysis and validation

If the loan is cleared after m months, time complexity is O(m). The program uses a fixed number of variables, so extra space complexity is O(1). With the 360-month safety limit, the maximum number of loop iterations is 360. `gcc -std=c11 -Wall -Wextra q6.c -o q6 ./q6`

# Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

## A. Diagnose the bottleneck (2 Marks)

The original prime-checking program tests every integer from 2 to n  1. For a large prime number, it performs almost n checks even though many of those checks are unnecessary. This repeated modulo operation is the main performance bottleneck.

For example, checking 97 in the original program requires 95 divisor checks. The square-root improvement reduces this to 8 checks, but it still checks even divisors after checking 2.

## B. State the objective (2 Marks)

The objective is to reduce the number of modulo operations while preserving the correct prime/non-prime result. The redesigned program will:

1. Handle numbers less than or equal to 1 as not prime.
2. Check divisibility by 2 once.
3. If the number is not divisible by 2, test only odd divisors from 3 up to the square root.
4. Stop immediately when a divisor is found.

## C. Revised solution and complete C program (4 Marks)

After checking 2, every remaining possible prime divisor is odd. Therefore, the loop increases the divisor by 2 instead of 1. The condition i <= n / i avoids possible overflow from calculating i * i. `#include`

```
<stdio.h>

int main() {     int n;
int isPrime =
```

```
1;     int checks =
0;    printf("Enter a number:

");

scanf("%d", &n);

    if (n <= 1)     {
isPrime =
0;     }     else if (n == 2)     {
isPrime = 1;     }     else if (n % 2 ==
0)     {          checks++;
isPrime = 0;     }     else     {
checks++;  /* Check divisor 2 */

        for (int i = 3; i <= n / i; i += 2)
{             checks++;

            if (n % i == 0)
{                 isPrime =
0;                 break;
}
        }
}

    if (isPrime)         printf("%d is
prime\\n", n);     else         printf("%d is

not prime\\n", n);     printf("Divisor checks

= %d\\n", checks);     return 0;

}
```

**Note about the counter:** For odd n greater than 2, the program counts the check for divisor 2 and then counts each odd divisor tested. For even n greater than 2, it counts the single successful divisibility test by 2. For n = 2 or n <= 1, no divisor test is needed.

## D. Predict the result (1 Mark)

For input 97, the program checks 2, 3, 5, 7 and 9. None divides 97, so the output is that 97 is prime. There are 5 divisor checks under the counter definition above.

```
Enter a number: 97 97
is prime
Divisor checks = 5
```

## E. Implement and validate (2 Marks)

Compile the original and redesigned programs using the same compiler settings. Run both programs with identical test inputs. Record the result and the number of divisor checks. The expected counts below use the counting convention stated above.

| Input | Original Q5 checks | Redesigned checks | Result |
|---|---|---|---|
| 97 | 95 | 5 | Prime |
| 100 | 1 | 1 | Not prime |
| 7919 | 7917 | 44 | Prime |
| 1000003 | 1000001 | 500 | Prime |

For prime inputs, the redesigned method reduces the number of checks by approximately half compared with the squareroot method, because it skips even divisors. The improvement is much larger compared with the original n  1 method.

**Validation cases:** Also test 1, 2, 3, 4 and 9. Expected results are: 1 not prime; 2 prime; 3 prime; 4 not prime; 9 not prime.

```
gcc -std=c11 -Wall -Wextra original_prime.c -o original_prime gcc -std=c11
-Wall -Wextra redesigned_prime.c -o redesigned_prime ./original_prime
./redesigned_prime
```

Take screenshots of both source files, compiler output and the matching test runs. Do not claim the program was executed unless you actually run it and capture the output.

## F. Defend the redesign (2 Marks)

The redesigned algorithm is better because it avoids testing even divisors after checking 2. This reduces unnecessary modulo operations while keeping the same correct answer. It is easy to implement, uses only a few variables and does not require additional arrays.

The original and redesigned algorithms both use O( n) worst-case time if the original comparison is the square-root version; however, the redesigned version performs about half as many divisor checks for large odd prime inputs. Its extra space complexity remains O(1). Compared with the original Q5 method that checks up to n 1, the redesign improves worst-case time from O(n) to O( n).

The redesign is appropriate when checking individual numbers. If the task is to generate all prime numbers up to a large limit, the Sieve of Eratosthenes may be a better choice.